Course Outline: Object Representation in TypeScript

Title: Object Representation in TypeScript: From Syntax to Advanced Patterns **Learnpack:** + Representación de objetos en TS **Prerequisite:** Skill 8.2 - Object Modeling with Class Diagrams **Target Audience:** Beginning developers learning TypeScript object implementation **Total Lessons:** 23 **Estimated Total Length:** ~13,100 words **Format:** Mixed (43.5% hands-on)

Course Description

This course bridges the gap between conceptual object modeling and practical TypeScript implementation. Building on your understanding of object models and class diagrams from Day 10, you'll now learn how to represent and manipulate objects in TypeScript code.

You'll master the syntax for creating objects, explore different ways to access and modify properties, understand the crucial difference between mutable and immutable data, and learn to work with complex nested structures. By the end of this course, you'll confidently create, manipulate, and organize object-based data structures in TypeScript.

This practical foundation prepares you for communicating effectively with AI coding assistants when building interfaces (Day 12), as you'll understand how to describe object structures and data relationships clearly.

Course Philosophy

This course emphasizes:

- **Practical implementation over theory** - You've learned the concepts; now build with them
- **Pattern recognition before memorization** - Understanding why syntax works, not just how
- **Safe coding practices** - Learning to avoid common mutation pitfalls
- **Building incrementally** - From simple literal objects to complex nested structures

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Object Syntax Fundamentals	4	~2,100
02 - Accessing Object Properties	4	~2,200
03 - Methods and Object Behavior	3	~1,700
04 - Complex Object Structures	4	~2,400
05 - Mutability and References	4	~2,200
06 - Assessment and Integration	2	~1,300

Section	Lessons	Estimated Words
07 - Moving Forward	1	~450
Total	**23**	**~13,100**

Section 00: Welcome to TypeScript Objects

00.0 Welcome to TypeScript Objects 🕮 (~450 words)

Learning Objectives:

- Understand how this course builds on Day 10's conceptual modeling
- Identify the practical skills you'll develop
- Recognize the connection to AI-assisted development (Day 12)

Content Outline:

- Brief recap: From models to code
- What you'll learn in this course
- Why object representation matters in real development
- Preview of the hands-on journey ahead
- Connection to the broader 4Geeks curriculum

Transitions: In Day 10, you learned to model objects conceptually using class diagrams. Now you'll bring those models to life in TypeScript code. This practical skill is essential for communicating with AI coding assistants, which you'll explore in Day 12.

Key Principles:

- Objects in code reflect real-world entities and relationships
- TypeScript provides multiple ways to define and access object properties
- Understanding syntax enables better AI collaboration
- Practical patterns matter more than memorizing every option

Examples:

- Day 10 model → TypeScript interface
- Class diagram property → TypeScript object attribute
- Real-world entity (user, product) → Code representation

Section 01: Object Syntax Fundamentals

01.0 Interfaces vs Literal Objects 🕮 (~500 words)

Learning Objectives:

- Differentiate between interfaces and literal objects in TypeScript
- Understand when to use each approach
- Recognize the relationship between models and code structures

Content Outline:

- What is an interface? (Type blueprint)
- What is a literal object? (Actual data instance)
- Syntax comparison: Interface vs literal
- When to use interfaces (reusable types)
- When to use literals (one-off data)
- How Day 10's models become interfaces

Transitions: You've designed object models on paper. Now you'll see how those designs translate into two TypeScript constructs: interfaces (the blueprint) and literal objects (the actual data). Let's start with understanding the difference.

Key Principles:

- Interfaces define the shape; literals contain the data
- Interfaces are reusable type definitions
- Literal objects are concrete instances
- Both work together in TypeScript applications

Examples:

```
// Interface (blueprint)
interface User {
  name: string;
  age: number;
  email: string;
}

// Literal object (instance)
const user: User = {
  name: "Ana",
  age: 25,
  email: "ana@example.com"
};
```

01.1 Creating and Defining Objects 💻 (~550 words)

Learning Objectives:

- Create TypeScript interfaces from conceptual models
- Write literal objects that conform to interfaces
- Validate object structures against type definitions

Content Outline:

- Step-by-step interface creation
- Defining property types (string, number, boolean)
- Creating literal objects

- TypeScript's type checking in action
- Common syntax errors and how to fix them

Transitions: Now that you understand the difference between interfaces and literals, let's practice creating both. You'll transform a simple model into working TypeScript code.

Key Principles:

- Property names use camelCase convention
- Every property needs a type annotation
- Literal objects must match their interface shape
- TypeScript catches mismatches at compile time

Examples:

```
// From model to interface
interface Product {
  id: number;
  name: string;
  price: number;
  inStock: boolean;
}

// Creating instances
const laptop: Product = {
  id: 1,
  name: "MacBook Pro",
  price: 1999,
  inStock: true
};
```

Hands-On Exercise: Create an interface for a `Book` object with properties:

- `title` (string)
- `author` (string)
- `yearPublished` (number)
- `isAvailable` (boolean)

Then create three literal book objects using this interface.

Success Criteria:

- Interface compiles without errors
- All three objects match the interface structure
- Property names use proper camelCase
- All types are correctly specified

01.2 Properties and Values 📖 (~500 words)

Learning Objectives:

- Understand the attribute vs value relationship
- Identify property types by their values
- Recognize valid and invalid property assignments

Content Outline:

- Attribute = property name (the key)
- Value = property data (what's stored)
- Type inference from values
- Required vs optional properties
- Property ordering (doesn't matter in objects)

Transitions: You've created objects, but let's clarify the fundamental relationship: attributes (the labels) versus values (the data). Understanding this distinction helps you reason about object structure.

Key Principles:

- Attributes are the named slots in your object
- Values are the actual data stored in those slots
- TypeScript infers types from values
- Property order doesn't affect object equality
- Each attribute can only hold one value (not multiple)

Examples:

```
interface Person {
  firstName: string;    // attribute: firstName
  lastName: string;     // attribute: lastName
  age: number;         // attribute: age
}

const person: Person = {
  firstName: "Carlos",  // value: "Carlos"
  lastName: "López",    // value: "López"
  age: 30              // value: 30
};
```

01.3 Practice: Building Your First Objects 💻 (~550 words)

Learning Objectives:

- Apply syntax fundamentals to create complete objects
- Debug common object definition errors
- Build confidence with TypeScript object creation

Content Outline:

- Guided practice: Multiple object types
- Identifying and fixing syntax errors
- Working with different property types
- Creating objects for real-world scenarios
- Testing your objects in code

Transitions: Time to solidify these fundamentals through practice. You'll create several different object types, encounter common errors, and learn to fix them systematically.

Key Principles:

- Write the interface first (blueprint before data)
- Check property names match exactly (case-sensitive)
- Ensure every property has a value
- Use TypeScript's error messages as guides

Examples: Practice scenarios:

1. `Course` interface with name, code, credits, instructor
2. `Restaurant` interface with name, cuisine, rating, isOpen
3. `Movie` interface with title, director, year, genres

Hands-On Exercise: Create a `TodoItem` interface and three todo objects:

Interface requirements:

- `id`: unique number
- `task`: description string
- `completed`: boolean status
- `priority`: number (1-5)

Create three different todos representing tasks of varying priority and completion status.

Code Example:

```
interface TodoItem {
  id: number;
  task: string;
  completed: boolean;
  priority: number;
}

// Student creates three instances here
```

Success Criteria:

- Interface defines all four properties with correct types
- Three distinct todo objects created
- All objects compile without errors
- Properties use meaningful, realistic values

- Priorities are within 1-5 range

Section 02: Accessing Object Properties

02.0 Dot vs Bracket Notation 🕮 (~500 words)

Learning Objectives:

- Master both property access syntaxes
- Understand when each notation is required
- Recognize the strengths and limitations of each approach

Content Outline:

- Dot notation: `object.property` (standard access)
- Bracket notation: `object['property']` (dynamic access)
- When dot notation works (most cases)
- When bracket notation is required (dynamic keys, special characters)
- Performance considerations (both are equally fast)

Transitions: You can create objects, but how do you access their data? TypeScript offers two ways: dot notation (simple and common) and bracket notation (powerful and flexible). Let's explore when to use each.

Key Principles:

- Dot notation is cleaner for known property names
- Bracket notation enables dynamic property access
- Bracket notation accepts variables as keys
- Both notations access the same data differently

Examples:

```
const user = {
  name: "Elena",
  age: 28,
  email: "elena@example.com"
};

// Dot notation (preferred when possible)
console.log(user.name);        // "Elena"
console.log(user.age);         // 28

// Bracket notation (required for variables)
const key = "email";
console.log(user[key]);        // "elena@example.com"

// Bracket notation (required for special characters)
const data = {
  "user-name": "Elena",        // hyphen requires brackets
  "first name": "Elena"        // space requires brackets
};
```

```
console.log(data["user-name"]); // Must use brackets
// console.log(data.user-name); // Error! Can't use dot
```

02.1 The Optional Chaining Operator 🕮 (~500 words)

Learning Objectives:

- Understand the problem optional chaining solves
- Use the `?.` operator safely
- Prevent runtime errors when properties might not exist

Content Outline:

- The problem: Accessing properties on undefined/null
- How traditional code handles missing data
- The optional chaining operator: `?.`
- Short-circuit behavior (stops at first undefined)
- Combining optional chaining with nested objects

Transitions: What happens when you try to access a property that doesn't exist? Your code crashes. The optional chaining operator (`?.`) is TypeScript's elegant solution to this common problem.

Key Principles:

- Optional chaining prevents runtime errors
- Returns `undefined` instead of throwing errors
- Especially useful for nested object access
- Use when property existence is uncertain
- Don't overuse on properties you know exist

Examples:

```
interface User {
  name: string;
  address?: {           // Optional property
    street: string;
    city: string;
  };
}

const user1: User = {
  name: "Ana",
  address: {
    street: "Main St",
    city: "Madrid"
  }
};
```

```
const user2: User = {
  name: "Carlos"
  // No address
};

// Without optional chaining (dangerous)
// console.log(user2.address.city); // Error! Cannot read property 'city' of
undefined

// With optional chaining (safe)
console.log(user1.address?.city);  // "Madrid"
console.log(user2.address?.city);  // undefined (no error)
```

02.2 Practice: Property Access Patterns 💻 (~600 words)

Learning Objectives:

- Apply both notation types appropriately
- Use optional chaining to prevent errors
- Access properties dynamically from user input or variables

Content Outline:

- Exercise 1: Static property access with dot notation
- Exercise 2: Dynamic property access with bracket notation
- Exercise 3: Safe access with optional chaining
- Exercise 4: Nested object traversal
- Common errors and debugging strategies

Transitions: Theory becomes skill through practice. You'll now work through scenarios requiring different access patterns, learning when to choose each approach.

Key Principles:

- Choose the simplest notation that works
- Use bracket notation when keys are dynamic
- Always use optional chaining for uncertain properties
- Test edge cases (missing properties, null values)

Examples: Scenarios to practice:

1. Product catalog with varying properties
2. User profiles with optional fields
3. Configuration objects accessed by variable keys
4. Nested API response data

Hands-On Exercise:

```typescript
interface Product {
  id: number;
  name: string;
  specs?: {
    weight?: number;
    dimensions?: {
      width: number;
      height: number;
      depth: number;
    };
  };
}

const products: Product[] = [
  {
    id: 1,
    name: "Laptop",
    specs: {
      weight: 2.5,
      dimensions: { width: 35, height: 25, depth: 2 }
    }
  },
  {
    id: 2,
    name: "Mouse",
    // No specs
  },
  {
    id: 3,
    name: "Keyboard",
    specs: {
      weight: 0.8
      // No dimensions
    }
  }
];

// Tasks:
// 1. Access each product's name using dot notation
// 2. Access specs.weight safely (may not exist)
// 3. Access dimensions.width safely (may not exist)
// 4. Create a function that accepts a property name as a variable
//    and returns that property from a product
```

Success Criteria:

- All product names accessed without errors
- Optional properties accessed safely (no crashes)
- Dynamic property function works with any valid property name
- Code handles missing nested properties gracefully
- Proper use of `?.` where properties might not exist

02.3 Common Access Mistakes 🕮 (~600 words)

Learning Objectives:

- Identify the most frequent property access errors
- Understand why certain patterns cause problems
- Learn to debug access-related issues systematically

Content Outline:

- Anti-pattern 1: Using variables with dot notation
- Anti-pattern 2: Assuming nested properties exist
- Anti-pattern 3: Properties with spaces or special characters
- Anti-pattern 4: Case sensitivity mismatches
- How to debug access errors using TypeScript's messages

Transitions: Now that you know the correct patterns, let's examine common mistakes developers make. Understanding these anti-patterns helps you write more robust code and debug faster.

Key Principles:

- Dot notation cannot use variables
- Always verify nested property existence
- Property names are case-sensitive
- Special characters require bracket notation
- TypeScript's error messages point to the problem

Examples:

Anti-Pattern 1: Variable Confusion

```
const user = { name: "Ana", age: 25 };
const key = "name";

// ❌ Wrong: Tries to access property literally named "key"
console.log(user.key);          // undefined

// ☑ Correct: Use brackets for variables
console.log(user[key]);         // "Ana"
```

Anti-Pattern 2: Unsafe Nested Access

```
interface User {
  profile?: {
    avatar?: string;
  };
}
```

```
const user: User = { /* no profile */ };

// ❌ Wrong: Crashes if profile is undefined
// console.log(user.profile.avatar);    // Error!

// ☑ Correct: Use optional chaining
console.log(user.profile?.avatar);     // undefined (safe)
```

Anti-Pattern 3: Special Characters

```
// ❌ Wrong: Special characters without brackets
const data = {
  "user-name": "Elena",
  "first name": "Elena"
};

// These won't work:
// console.log(data.user-name);    // Error! Tries to subtract
// console.log(data.first name);   // Error! Syntax error

// ☑ Correct: Use bracket notation
console.log(data["user-name"]);    // "Elena"
console.log(data["first name"]);   // "Elena"
```

Anti-Pattern 4: Case Sensitivity

```
const product = {
  productName: "Laptop",
  ProductPrice: 999
};

// ❌ Wrong: Case mismatch
console.log(product.productname);   // undefined (lowercase 'n')
console.log(product.productprice);  // undefined (lowercase 'p')

// ☑ Correct: Match exact case
console.log(product.productName);   // "Laptop"
console.log(product.ProductPrice);  // 999
```

Debugging Strategy:

1. Check TypeScript's error message (usually points to the exact issue)
2. Verify property name spelling and case
3. Confirm the property actually exists on the object
4. Use console.log to inspect the object structure
5. Test with optional chaining if property might not exist

Section 03: Methods and Object Behavior

03.0 Understanding Methods in Objects 🕮 (~500 words)

Learning Objectives:

- Differentiate between properties and methods
- Understand methods as functions stored in objects
- Recognize when to use methods vs standalone functions

Content Outline:

- What is a method? (Property that holds a function)
- Syntax for defining methods
- Methods vs properties: Behavior vs data
- The `this` keyword (brief introduction, not deep dive)
- When to use methods (behavior belongs to the object)

Transitions: So far, your objects have stored data (properties). But objects can also have behavior (methods). Methods are functions that belong to an object, allowing objects to perform actions on their own data.

Key Principles:

- Methods are properties that contain functions
- Methods define what an object can do
- Methods can access other properties in the same object
- Methods make objects self-contained and reusable

Examples:

```
interface Calculator {
  value: number;
  add: (n: number) => number;
  reset: () => void;
}

const calculator: Calculator = {
  value: 0,

  add: function(n: number): number {
    this.value += n;
    return this.value;
  },

  reset: function(): void {
    this.value = 0;
  }
};

calculator.add(5);        // value becomes 5
calculator.add(3);        // value becomes 8
```

```
console.log(calculator.value);  // 8
calculator.reset();       // value becomes 0
```

03.1 Implementing Object Methods 💻 (~600 words)

Learning Objectives:

- Create interfaces with method signatures
- Implement methods in literal objects
- Call methods and use their return values

Content Outline:

- Defining method signatures in interfaces
- Implementing methods in objects
- Arrow function vs traditional function syntax
- Calling methods and capturing results
- Methods that modify vs methods that return

Transitions: Let's put methods into practice. You'll create objects that not only store data but also perform operations on that data through methods.

Key Principles:

- Method signatures specify parameter types and return types
- Methods can modify object properties (mutations)
- Methods can return computed values
- Keep methods focused on single responsibilities

Examples:

```
interface BankAccount {
  balance: number;
  deposit: (amount: number) => number;
  withdraw: (amount: number) => number;
  getBalance: () => number;
}

const account: BankAccount = {
  balance: 1000,

  deposit: function(amount: number): number {
    this.balance += amount;
    return this.balance;
  },

  withdraw: function(amount: number): number {
    if (amount <= this.balance) {
      this.balance -= amount;
    }
```

```
    return this.balance;
  },

  getBalance: function(): number {
    return this.balance;
  }
};
```

Hands-On Exercise:

Create a `ShoppingCart` interface and object:

Interface requirements:

- `items`: array of product names (string[])
- `addItem`: method that adds a product name to items
- `removeItem`: method that removes a product name from items
- `getItemCount`: method that returns the number of items
- `clear`: method that empties the cart

Implementation requirements:

- Start with an empty cart
- `addItem` should add to the array
- `removeItem` should remove the first matching item
- `getItemCount` should return the array length
- `clear` should empty the array

Code Template:

```
interface ShoppingCart {
  items: string[];
  addItem: (product: string) => void;
  removeItem: (product: string) => void;
  getItemCount: () => number;
  clear: () => void;
}

// Implement the cart object here
```

Success Criteria:

- Interface defines all properties and methods with correct types
- Object implements all methods
- `addItem` adds products to the array
- `removeItem` removes specified products
- `getItemCount` returns correct count
- `clear` empties the cart
- All methods work when called sequentially

03.2 Practice: Building Interactive Objects 💻 (~600 words)

Learning Objectives:

- Design objects with complex method interactions
- Implement validation logic in methods
- Create objects that maintain internal state correctly

Content Outline:

- Building a counter object with increment/decrement
- Creating a todo manager with add/complete/delete
- Implementing a timer object with start/stop/reset
- Validating method inputs
- Testing method behavior

Transitions: You understand individual methods. Now let's build objects where multiple methods work together, maintaining consistent internal state while providing useful functionality.

Key Principles:

- Methods should validate their inputs
- Methods should maintain object consistency
- Related methods should work together smoothly
- Return meaningful values or status indicators

Examples:

Hands-On Exercise 1: Counter

```
interface Counter {
  count: number;
  increment: () => number;
  decrement: () => number;
  reset: () => void;
  getValue: () => number;
}

// Create a counter that:
// - Starts at 0
// - increment() adds 1 and returns new value
// - decrement() subtracts 1 (but never goes below 0)
// - reset() sets back to 0
// - getValue() returns current count
```

Hands-On Exercise 2: Todo Manager

```
interface Todo {
  id: number;
```

```
  task: string;
  completed: boolean;
}

interface TodoManager {
  todos: Todo[];
  nextId: number;
  addTodo: (task: string) => Todo;
  completeTodo: (id: number) => boolean;
  deleteTodo: (id: number) => boolean;
  getTodos: () => Todo[];
  getCompletedCount: () => number;
}

// Create a todo manager that:
// - Maintains a list of todos
// - addTodo creates a new todo with auto-incrementing ID
// - completeTodo marks a todo as completed by ID
// - deleteTodo removes a todo by ID
// - getTodos returns all todos
// - getCompletedCount returns how many are completed
```

Success Criteria:

- Counter never goes below zero
- Counter methods return correct values
- Todo manager assigns unique IDs
- Todos can be marked complete by ID
- Todos can be deleted by ID
- Methods return success/failure status appropriately
- All objects maintain consistent state

Section 04: Complex Object Structures

04.0 Working with Nested Objects 🕮 (~550 words)

Learning Objectives:

- Understand nested object structures
- Access deeply nested properties safely
- Design interfaces for hierarchical data

Content Outline:

- What are nested objects? (Objects within objects)
- Common use cases (addresses, preferences, metadata)
- Defining nested structures in interfaces
- Accessing nested properties with dot notation
- Using optional chaining for nested access

Transitions: Real-world data is rarely flat. User profiles have addresses, products have specifications, configurations have multiple levels. Let's learn to work with nested object structures.

Key Principles:

- Nested objects represent hierarchical relationships
- Each level can have its own interface
- Optional chaining is essential for nested access
- Design nested structures to match real-world organization

Examples:

```
interface Address {
  street: string;
  city: string;
  postalCode: string;
  country: string;
}

interface User {
  name: string;
  email: string;
  address: Address;           // Nested object
  preferences?: {             // Optional nested object
    newsletter: boolean;
    theme: string;
  };
}

const user: User = {
  name: "Laura",
  email: "laura@example.com",
  address: {
    street: "Gran Vía 1",
    city: "Madrid",
    postalCode: "28013",
    country: "España"
  },
  preferences: {
    newsletter: true,
    theme: "dark"
  }
};

// Accessing nested properties
console.log(user.address.city);              // "Madrid"
console.log(user.preferences?.theme);        // "dark"
console.log(user.settings?.language);        // undefined (safe)
```

04.1 Arrays of Objects 🕮 (~550 words)

Learning Objectives:

- Create and manage arrays containing objects
- Iterate through object arrays
- Filter and transform object collections

Content Outline:

- Arrays of objects syntax
- Common patterns (lists of users, products, transactions)
- Accessing object properties in arrays
- Iterating with forEach, map, filter
- Finding specific objects in arrays

Transitions: Individual objects are useful, but real applications work with collections. An array of objects combines the structure of objects with the power of array operations.

Key Principles:

- Arrays of objects represent collections of similar items
- Each element is a complete object
- Array methods work perfectly with object collections
- Maintain consistent object structure across the array

Examples:

```
interface Product {
  id: number;
  name: string;
  price: number;
  category: string;
}

const products: Product[] = [
  { id: 1, name: "Laptop", price: 999, category: "Electronics" },
  { id: 2, name: "Mouse", price: 25, category: "Electronics" },
  { id: 3, name: "Desk", price: 350, category: "Furniture" }
];

// Accessing properties
console.log(products[0].name);           // "Laptop"

// Iterating
products.forEach(product => {
  console.log(`${product.name}: $${product.price}`);
});

// Filtering
const electronics = products.filter(p => p.category === "Electronics");

// Mapping
const names = products.map(p => p.name);
```

```
// ["Laptop", "Mouse", "Desk"]

// Finding
const laptop = products.find(p => p.name === "Laptop");
```

04.2 Practice: Managing Complex Data 💻 (~650 words)

Learning Objectives:

- Build nested object structures from requirements
- Manipulate arrays of objects effectively
- Combine nested objects and arrays

Content Outline:

- Exercise: User profiles with addresses and orders
- Exercise: Product catalog with categories and reviews
- Accessing deeply nested data safely
- Updating nested properties
- Common pitfalls with complex structures

Transitions: Time to work with realistic, complex data structures. You'll build systems that combine nested objects and arrays, reflecting real-world application data.

Key Principles:

- Design structures that mirror real relationships
- Use optional chaining for uncertain nested paths
- Keep interfaces well-organized and readable
- Test access patterns before building complex logic

Hands-On Exercise:

Create an e-commerce order system:

```
interface Product {
  id: number;
  name: string;
  price: number;
}

interface OrderItem {
  product: Product;
  quantity: number;
}

interface ShippingAddress {
  street: string;
  city: string;
  postalCode: string;
```

```
}

interface Order {
  orderId: string;
  customer: {
    name: string;
    email: string;
  };
  items: OrderItem[];
  shippingAddress: ShippingAddress;
  total: number;
}

// Tasks:
// 1. Create an order with at least 2 items
// 2. Calculate the total (price * quantity for each item)
// 3. Access the customer's email
// 4. Access the shipping city
// 5. Count how many items are in the order
// 6. List all product names in the order
```

Success Criteria:

- Order structure matches all interface requirements
- Total is correctly calculated
- All nested properties accessible
- Can extract specific nested data (customer email, shipping city)
- Can iterate through order items
- No errors when accessing nested properties

04.3 Object Composition Patterns 💻 (~650 words)

Learning Objectives:

- Combine multiple objects into larger structures
- Reuse interface definitions across structures
- Build modular, maintainable object hierarchies

Content Outline:

- Composition vs repetition
- Reusing interfaces in multiple contexts
- Building objects from smaller components
- Practical patterns (user with roles, product with variants)
- Maintaining type safety in compositions

Transitions: Rather than creating massive objects from scratch, experienced developers compose them from reusable parts. Let's learn this modular approach to object design.

Key Principles:

- Define small, focused interfaces
- Compose larger structures from smaller ones
- Reuse interface definitions wherever possible
- Keep each interface single-purpose

Examples:

Composition Example:

```
// Small, reusable interfaces
interface Timestamp {
  createdAt: Date;
  updatedAt: Date;
}

interface Author {
  id: number;
  name: string;
  email: string;
}

interface Content {
  title: string;
  body: string;
}

// Composed interfaces
interface BlogPost extends Timestamp {
  id: number;
  author: Author;
  content: Content;
  tags: string[];
}

interface Comment extends Timestamp {
  id: number;
  author: Author;
  content: Content;
  postId: number;
}

// Both BlogPost and Comment reuse Author, Content, and Timestamp
```

Hands-On Exercise:

Build a course management system using composition:

```
// Define these small interfaces:
// - Person (name, email)
// - Duration (hours, minutes)
```

```
// - DateRange (startDate, endDate)

// Then compose into:
// - Student (extends Person, adds studentId, enrolledCourses)
// - Instructor (extends Person, adds bio, courses)
// - Course (name, code, instructor, duration, schedule, students)

// Tasks:
// 1. Define the three base interfaces
// 2. Define the three composed interfaces
// 3. Create sample data:
//    - 2 students
//    - 1 instructor
//    - 1 course with both students enrolled
// 4. Access nested data (instructor email, student names, course duration)
```

Success Criteria:

- Base interfaces are reusable and focused
- Composed interfaces properly extend base interfaces
- No duplicate interface definitions
- Sample data uses all interfaces correctly
- Can access all nested properties
- Type safety maintained throughout

Section 05: Mutability and References

05.0 Understanding Mutability 🕮 (~500 words)

Learning Objectives:

- Understand what mutability means in programming
- Differentiate between mutable and immutable data
- Recognize when objects can be modified

Content Outline:

- Definition: Mutability vs immutability
- Real-world analogy: Notebook (mutable) vs book (immutable)
- Primitive types are immutable (numbers, strings, booleans)
- Objects and arrays are mutable
- Why mutability matters

Transitions: You've created and modified objects freely, but there's a crucial concept underlying all object manipulation: mutability. Understanding whether data can change is fundamental to avoiding bugs.

Key Principles:

- Mutable = can be changed after creation
- Immutable = cannot be changed after creation

- Primitives (string, number, boolean) are immutable
- Objects and arrays are mutable
- Mutation can have unexpected side effects

Examples:

Immutable (Primitives):

```
let name = "Ana";
let uppercased = name.toUpperCase();

console.log(name);        // "Ana" (original unchanged)
console.log(uppercased);  // "ANA" (new value created)

let num = 5;
let doubled = num * 2;

console.log(num);         // 5 (original unchanged)
console.log(doubled);     // 10 (new value created)
```

Mutable (Objects):

```
const user = {
  name: "Ana",
  age: 25
};

user.age = 26;            // Mutation! Object changed
console.log(user.age);    // 26 (original modified)

const numbers = [1, 2, 3];
numbers.push(4);          // Mutation! Array changed
console.log(numbers);     // [1, 2, 3, 4] (original modified)
```

Real-World Analogy:

- **Book (Immutable):** Published books can't be changed. If you find a typo, you need a new edition (new value).
- **Notebook (Mutable):** You can erase and rewrite in a notebook. The same notebook changes over time.

05.1 Pass by Reference vs Pass by Value 🕮 (~550 words)

Learning Objectives:

- Understand how JavaScript handles primitive vs object assignments
- Recognize when you're working with a reference vs a copy
- Predict the behavior of assignments and function parameters

Content Outline:

- Pass by value (primitives create copies)
- Pass by reference (objects share the same memory)
- Assignment behavior differences
- Function parameter behavior
- Implications for object mutation

Transitions: Mutability leads to a crucial question: when you assign an object to a variable, are you creating a copy or sharing the original? The answer depends on whether it's a primitive or an object.

Key Principles:

- Primitives pass by value (copy the data)
- Objects pass by reference (share the data)
- Multiple variables can reference the same object
- Changing a reference affects all variables pointing to it
- This behavior applies to function parameters too

Examples:

Pass by Value (Primitives):

```
let a = 10;
let b = a;          // b gets a COPY of a's value

b = 20;             // Only b changes

console.log(a);     // 10 (unchanged)
console.log(b);     // 20 (changed)
```

Pass by Reference (Objects):

```
const user1 = {
  name: "Ana",
  age: 25
};

const user2 = user1;    // user2 references the SAME object

user2.age = 30;         // Changes the shared object

console.log(user1.age); // 30 (changed!)
console.log(user2.age); // 30 (changed!)
// Both variables point to the same object
```

Function Parameters:

```
function changeAge(person: { name: string; age: number }) {
  person.age = 100;     // Mutates the original object
}

const user = { name: "Carlos", age: 25 };
changeAge(user);        // Passes reference to user

console.log(user.age);  // 100 (original object was mutated!)
```

Visual Explanation:

```
Primitives:
a = 10  [memory: 10]
b = a   [memory: 10]  ← Different memory locations

Objects:
user1 = {...}  [memory: 0x001 → {name: "Ana", age: 25}]
user2 = user1  [memory: 0x001 → {name: "Ana", age: 25}]
                        ↑
                 Same memory location!
```

05.2 Practice: Reference Behavior 💻 (~600 words)

Learning Objectives:

- Predict the outcome of reference assignments
- Create true copies of objects when needed
- Avoid unintentional mutations

Content Outline:

- Predicting reference behavior
- Creating object copies (shallow with spread operator)
- Understanding when copies are needed
- Testing reference vs copy scenarios
- Common mutation bugs

Transitions: Theory meets practice. You'll work through scenarios that expose reference behavior, learning to control when objects share memory vs when they're independent.

Key Principles:

- Test assumptions by logging before and after
- Use spread operator `{...obj}` to create shallow copies
- Shallow copies only copy the first level
- Deep nested objects still share references
- Think before mutating: do I want to affect the original?

Examples:

Spread Operator (Shallow Copy):

```
const original = {
  name: "Ana",
  age: 25
};

const copy = { ...original };  // Shallow copy

copy.age = 30;

console.log(original.age);     // 25 (unchanged)
console.log(copy.age);         // 30 (changed)
```

Hands-On Exercise:

Part 1: Predict the Output

```
// What will be logged?

const product1 = {
  name: "Laptop",
  price: 999
};

const product2 = product1;
product2.price = 1299;

console.log(product1.price);  // Your prediction: ?
console.log(product2.price);  // Your prediction: ?

// Now create a copy
const product3 = { ...product1 };
product3.price = 799;

console.log(product1.price);  // Your prediction: ?
console.log(product3.price);  // Your prediction: ?
```

Part 2: Fix the Mutation Bug

```
interface Settings {
  theme: string;
  notifications: boolean;
}

function updateTheme(settings: Settings, newTheme: string): Settings {
```

```
  settings.theme = newTheme;   // Bug! Mutates original
  return settings;
}

const userSettings: Settings = {
  theme: "light",
  notifications: true
};

const newSettings = updateTheme(userSettings, "dark");

console.log(userSettings.theme);  // "dark" (oops! Original changed)

// Fix this function to return a new object without mutating the original
```

Part 3: Array of Objects

```
const users = [
  { name: "Ana", score: 100 },
  { name: "Carlos", score: 85 }
];

const updatedUsers = users;  // What happens here?
updatedUsers[0].score = 150;

console.log(users[0].score);  // Your prediction: ?

// Create a true copy of the array with copied objects
```

Success Criteria:

- Correctly predict all reference behavior
- Fix mutation bug using spread operator
- Create true copies of arrays and their objects
- Understand when mutations affect other variables
- Can explain why each scenario behaves as it does

05.3 Avoiding Mutation Pitfalls 🕮 (~550 words)

Learning Objectives:

- Identify common mutation anti-patterns
- Understand the risks of unintended mutations
- Learn strategies to prevent mutation bugs

Content Outline:

- Anti-pattern 1: Unintentional shared state
- Anti-pattern 2: Mutating function parameters

- Anti-pattern 3: Shallow copy assumptions with nested objects
- Anti-pattern 4: Array method mutations (push, splice, etc.)
- Best practices: When to mutate vs when to copy

Transitions: Understanding references is one thing; avoiding their pitfalls is another. Let's examine common mistakes developers make with object mutations and learn to prevent them.

Key Principles:

- Default to immutability (create new objects)
- Mutate only when explicitly intended
- Be especially careful with function parameters
- Remember: shallow copies don't protect nested objects
- Use const, but know it doesn't prevent property changes

Examples:

Anti-Pattern 1: Unintentional Shared State

```
// ❌ Wrong: All users share the same settings object
const defaultSettings = { theme: "light", lang: "en" };

const user1 = { name: "Ana", settings: defaultSettings };
const user2 = { name: "Carlos", settings: defaultSettings };

user1.settings.theme = "dark";

console.log(user2.settings.theme);  // "dark" (oops!)

// ☑ Correct: Each user gets their own settings copy
const user1 = {
  name: "Ana",
  settings: { ...defaultSettings }
};
const user2 = {
  name: "Carlos",
  settings: { ...defaultSettings }
};
```

Anti-Pattern 2: Mutating Parameters

```
// ❌ Wrong: Function mutates the original
function addDiscount(product: Product): Product {
  product.price *= 0.9;  // Mutates original!
  return product;
}

// ☑ Correct: Function returns new object
function addDiscount(product: Product): Product {
  return {
```

```
    ...product,
    price: product.price * 0.9
  };
}
```

Anti-Pattern 3: Shallow Copy with Nested Objects

```
const user = {
  name: "Ana",
  address: {
    city: "Madrid",
    country: "Spain"
  }
};

// ❌ Shallow copy doesn't protect nested objects
const copy = { ...user };
copy.address.city = "Barcelona";

console.log(user.address.city);  // "Barcelona" (oops! Still shared)

// ☑ Deep copy needed for nested objects
const copy = {
  ...user,
  address: { ...user.address }
};
```

Anti-Pattern 4: Array Mutation Methods

```
const numbers = [1, 2, 3];

// ❌ These mutate the original array:
numbers.push(4);        // Mutates
numbers.splice(0, 1);   // Mutates
numbers.sort();         // Mutates

// ☑ These create new arrays:
const added = [...numbers, 4];
const filtered = numbers.filter(n => n > 1);
const mapped = numbers.map(n => n * 2);
```

When to Mutate vs Copy:

Mutate when:

- Working with a single local object
- Performance is critical (large datasets)
- Explicitly managing state (with clear intent)

Copy when:

- Returning from functions
- Updating application state
- Sharing data between components
- In doubt (safer default)

Section 06: Assessment and Integration

06.0 Final Project: Object-Oriented Data Modeling 💻 (~700 words)

Learning Objectives:

- Apply all learned concepts in a comprehensive project
- Design and implement a multi-object system
- Demonstrate mastery of interfaces, methods, and complex structures

Content Outline:

- Project specification
- Planning the object structure
- Implementing interfaces and objects
- Testing functionality
- Handling edge cases

Transitions: You've learned individual concepts. Now synthesize everything into a complete system that combines interfaces, methods, nested structures, and safe property access.

Key Principles:

- Plan before coding (interfaces first)
- Build incrementally (test each piece)
- Use composition (reusable interfaces)
- Handle edge cases (optional chaining, validation)

Hands-On Project: Library Management System

Requirements:

Create a library management system with the following features:

1. Book Interface:

- `isbn`: string (unique identifier)
- `title`: string
- `author`: Author object (nested)
- `year`: number
- `available`: boolean
- `borrower?`: Borrower object (optional, when checked out)

2. Author Interface:

- `name`: string
- `birthYear`: number
- `nationality`: string

3. Borrower Interface:

- `id`: number
- `name`: string
- `email`: string
- `borrowedBooks`: Book[] (array of books)

4. Library Interface:

- `books`: Book[] (all books in library)
- `borrowers`: Borrower[] (registered borrowers)
- `addBook`: method to add a book
- `registerBorrower`: method to register a borrower
- `checkoutBook`: method to lend a book (by ISBN to borrower ID)
- `returnBook`: method to return a book (by ISBN)
- `searchByTitle`: method to find books by title
- `getAvailableBooks`: method to get all available books
- `getBorrowerBooks`: method to get all books borrowed by a specific borrower

Implementation Tasks:

1. **Define all interfaces** with proper types
2. **Create a Library object** with:
    - At least 5 books (with different authors)
    - At least 3 registered borrowers
    - All required methods implemented
3. **Implement business logic:**
    - `checkoutBook`: Verify book exists and is available before lending
    - `checkoutBook`: Add book to borrower's borrowedBooks array
    - `returnBook`: Set book as available and remove from borrower's list
    - `searchByTitle`: Case-insensitive partial match
    - Validate inputs in all methods
4. **Test scenarios:**
    - Checkout an available book
    - Try to checkout an unavailable book (should fail gracefully)
    - Return a book
    - Search for books by partial title
    - List all books for a specific borrower

Code Template:

```typescript
interface Author {
  // Define properties
}

interface Borrower {
  // Define properties
}

interface Book {
  // Define properties
}

interface Library {
  // Define properties and methods
}

const library: Library = {
  books: [
    // Add at least 5 books
  ],
  borrowers: [
    // Add at least 3 borrowers
  ],

  addBook: function(book: Book): void {
    // Implementation
  },

  registerBorrower: function(borrower: Borrower): void {
    // Implementation
  },

  checkoutBook: function(isbn: string, borrowerId: number): boolean {
    // Implementation
    // Return true if successful, false if failed
  },

  returnBook: function(isbn: string): boolean {
    // Implementation
  },

  searchByTitle: function(title: string): Book[] {
    // Implementation
  },

  getAvailableBooks: function(): Book[] {
    // Implementation
  },

  getBorrowerBooks: function(borrowerId: number): Book[] {
    // Implementation
  }
};
```

```
// Test your implementation
console.log("Available books:", library.getAvailableBooks().length);
library.checkoutBook("978-0-123456-78-9", 1);
console.log("After checkout:", library.getAvailableBooks().length);
```

Success Criteria:

- All interfaces defined with correct types
- Library object implements all methods
- Checkout validates availability before lending
- Return method updates book and borrower state
- Search works with partial, case-insensitive matches
- Methods handle edge cases (book not found, borrower not found)
- No runtime errors when testing all scenarios
- Code uses optional chaining where appropriate
- Methods maintain data consistency (no orphaned references)

Bonus Challenges:

- Implement a `getDueBooks` method (books borrowed over 14 days ago)
- Add a `renewCheckout` method to extend a borrowing period
- Implement overdue fine calculation

06.1 Knowledge Check 🧐 (~600 words)

Learning Objectives:

- Assess understanding of TypeScript object concepts
- Demonstrate ability to analyze object code
- Identify correct patterns and anti-patterns

Question Format: Scenario-based (code analysis and problem-solving)

Questions:

Question 1: Interface vs Literal Object

```
// Analyze this code:

interface Product {
  name: string;
  price: number;
}

const laptop = {
  name: "MacBook",
  price: 1999,
  color: "Silver"
```

```
};

const desktop: Product = laptop;
```

What happens when this code runs? A) Compiles successfully, color property is ignored B) Compilation error: laptop has extra property 'color' C) Runtime error when accessing color D) desktop.color is accessible

Correct Answer: A **Explanation:** TypeScript allows excess properties in object literals when assigning to a typed variable, as long as all required properties exist. The extra property is ignored for type checking purposes.

Question 2: Property Access

```
const user = {
  name: "Ana",
  "user-id": 123,
  preferences: {
    theme: "dark"
  }
};

const key = "name";
```

Which of these will successfully access the user's name? A) `user.key` B) `user[key]` C) `user.name` D) Both B and C

Correct Answer: D **Explanation:** `user.name` uses dot notation for known properties. `user[key]` uses bracket notation with a variable. Both work. `user.key` would try to access a property literally named "key".

Question 3: Optional Chaining

```
interface User {
  name: string;
  address?: {
    city: string;
    postal?: {
      code: string;
    };
  };
}

const user: User = {
  name: "Carlos"
};

console.log(user.address?.postal?.code);
```

What gets logged? A) Runtime error B) Compilation error C) undefined D) null

Correct Answer: C **Explanation:** Optional chaining short-circuits at the first undefined property. Since `address` is undefined, the chain stops and returns undefined (not an error).

Question 4: Reference Behavior

```
const settings = {
  theme: "light",
  lang: "en"
};

function changeTheme(config) {
  config.theme = "dark";
  return config;
}

const newSettings = changeTheme(settings);
console.log(settings.theme);
```

What gets logged? A) "light" B) "dark" C) undefined D) Compilation error

Correct Answer: B **Explanation:** Objects are passed by reference. The function mutates the original `settings` object, so `settings.theme` is now "dark".

Question 5: Creating Copies

```
const original = {
  name: "Ana",
  scores: [95, 87, 92]
};

const copy = { ...original };
copy.scores.push(88);

console.log(original.scores.length);
```

What is the length of `original.scores`? A) 3 (unchanged) B) 4 (changed) C) undefined D) Compilation error

Correct Answer: B **Explanation:** Spread operator creates a shallow copy. The `scores` array is still shared between original and copy. Pushing to `copy.scores` also affects `original.scores`.

Question 6: Methods in Objects

```
const counter = {
  count: 0,
```

```
  increment: function() {
    this.count++;
  },
  getValue: () => {
    return this.count;
  }
};

counter.increment();
console.log(counter.getValue());
```

What gets logged? A) 1 B) 0 C) undefined D) NaN

Correct Answer: C **Explanation:** Arrow functions don't have their own `this` context. `getValue` as an arrow function doesn't access `counter.count` correctly. Traditional function syntax should be used for methods that need `this`.

Question 7: Complex Structures

```
interface Order {
  items: Array<{
    product: string;
    quantity: number;
  }>;
}

const order: Order = {
  items: [
    { product: "Laptop", quantity: 1 },
    { product: "Mouse", quantity: 2 }
  ]
};

const total = order.items.reduce((sum, item) => sum + item.quantity, 0);
```

What is the value of `total`? A) 2 B) 3 C) undefined D) Compilation error

Correct Answer: B **Explanation:** `reduce` sums all quantities: 1 + 2 = 3.

Evaluation Criteria:

- 7/7 correct: Excellent understanding
- 5-6 correct: Good understanding, review specific topics
- 3-4 correct: Fair understanding, practice recommended
- 0-2 correct: Review course material before proceeding

Score Interpretation:

- **Excellent (7/7):** You've mastered TypeScript objects. Ready for Day 12.
- **Good (5-6):** Strong foundation with minor gaps. Review questions missed.
- **Fair (3-4):** Understand basics but need more practice. Revisit hands-on lessons.
- **Needs Review (0-2):** Core concepts need reinforcement. Review course from Section 01.

Section 07: Moving Forward with TypeScript Objects

07.0 Your Journey with TypeScript Objects (~450 words)

Content Outline:

- What you've accomplished in this course
- How object skills connect to Day 12 (AI communication)
- Real-world applications of these concepts
- Resources for continued learning
- Next steps in your 4Geeks journey

Course Recap:

You began this course understanding object models conceptually (Day 10). Now you can:

☑ **Create structured data** with interfaces and literal objects ☑ **Access properties** safely using dot notation, brackets, and optional chaining
☑ **Build object behavior** with methods that maintain internal state ☑ **Work with complexity** through nested objects and arrays of objects ☑ **Manage mutations** by understanding references and creating copies when needed

From Concepts to Code:

Day 10 taught you to *think* in objects (models, diagrams, relationships). Today you learned to *build* in objects (syntax, methods, structures). This combination—conceptual understanding plus practical implementation—makes you effective with TypeScript.

Connection to AI Communication (Day 12):

Tomorrow you'll learn to communicate effectively with AI coding assistants for building interfaces. The object knowledge you've gained is crucial:

- **Describing data structures** to AI requires understanding objects
- **Requesting nested components** means knowing nested object patterns
- **Specifying props and state** uses the same concepts as object properties
- **Debugging AI-generated code** requires reading object structures confidently

Real-World Applications:

These object skills apply to every TypeScript/JavaScript application:

- **Frontend state management:** User profiles, shopping carts, form data
- **API responses:** Working with JSON data from backend services
- **Component props:** Passing data between React/Vue components

- **Configuration objects:** App settings, feature flags, user preferences
- **Data modeling:** Representing business entities (users, products, orders)

Best Practices to Remember:

1. **Design interfaces first** (blueprint before data)
2. **Use optional chaining** for uncertain properties
3. **Prefer immutability** (create new objects, don't mutate)
4. **Compose from small pieces** (reusable interfaces)
5. **Validate in methods** (protect object consistency)

Resources for Continued Learning:

- **TypeScript Handbook:** Official documentation on interfaces and types
- **JavaScript.info:** Deep dives into object behavior and prototypes
- **Practice:** Mastering JS learnpack exercises (arrays, objects, methods)
- **4Geeks Community:** Slack channels for questions and discussions

Your Next Steps:

1. **Review the final project:** Ensure you understand every part of the library system
2. **Complete any skipped exercises:** Hands-on practice builds muscle memory
3. **Prepare for Day 12:** You're ready to communicate with AI about interfaces
4. **Stay curious:** Objects are fundamental—you'll use them every day

Celebration:

You've transformed from conceptual understanding to practical implementation. This isn't just theory—you can now build real data structures that power applications. That's significant progress in your journey to becoming an AI-assisted developer.

Tomorrow: You'll learn to leverage your object knowledge by communicating effectively with AI to build user interfaces. See you in Day 12!

End of Course Outline